OBJETIVO

Comparar experimentalmente el rendimiento de un algoritmo de ordenamiento de complejidad cuadrática $O(n^2)$, utilizando Burbuja, frente a un algoritmo de complejidad promedio O(n log n), utilizando QuickSort implementado en C#.

La comparación se realiza mediante diferentes tamaños de entrada, siguiendo un protocolo de calentamiento, tres corridas por prueba y cálculo del promedio de los tiempos obtenidos.

ALGORITMOS UTILIZADOS

Burbuja (Bubble Sort)

Burbuja recorre repetidamente el arreglo y compara elementos adyacentes. Cuando dos elementos se encuentran en el orden incorrecto, se intercambian. En el caso general, su complejidad temporal es $O(n^2)$, por lo que su tiempo de ejecución aumenta considerablemente cuando crece la cantidad de datos.

QuickSort

QuickSort selecciona un elemento denominado pivote y reorganiza el arreglo colocando los elementos menores o iguales al pivote a un lado y los mayores al otro. Después aplica el mismo procedimiento de manera recursiva sobre las particiones. Su complejidad promedio es O(n log n), aunque su peor caso puede llegar a $O(n^2)$, dependiendo de la elección del pivote y de los datos.

En esta práctica se utilizó como pivote el último elemento de la sección que se está particionando.

METODOLOGÍA

- Se desarrolló el programa en C# dentro del proyecto Practicas/C5.
- Se utilizaron arreglos de 10.000, 20.000, 40.000 y 80.000 elementos.
- Los datos fueron generados automáticamente mediante la clase Random.
- Para garantizar una comparación justa, cada algoritmo recibió una copia del mismo arreglo original.
- Antes de las mediciones se realizó una ejecución de calentamiento.
- Cada algoritmo fue ejecutado tres veces para cada tamaño de entrada.
- Se utilizó Stopwatch de C# para medir el tiempo de ejecución en milisegundos.
- Se calculó el promedio de las tres corridas.
- Después de cada ordenamiento se verificó que el arreglo estuviera correctamente ordenado.

PROTOCOLO DE MEDICIÓN

El protocolo utilizado para cada cantidad de elementos fue:

1. Generar los datos.

2. Realizar el calentamiento ejecutando ambos algoritmos una vez.
3. Ejecutar Burbuja tres veces y registrar cada tiempo.
4. Ejecutar QuickSort tres veces y registrar cada tiempo.
5. Calcular el promedio de las tres mediciones.
6. Comprobar que el arreglo resultante esté ordenado.

RESULTADOS OBTENIDOS

Los siguientes valores corresponden a las mediciones realizadas en el equipo utilizado para la práctica. Los tiempos están expresados en milisegundos (ms).

Elementos	Burbuja (ms)	QuickSort (ms)
10.000	679,56	2,04
20.000	2.185,46	7,93
40.000	8.622,79	11,01
80.000	34.856,32	14,90

ANÁLISIS DE LOS RESULTADOS

Los resultados muestran una diferencia marcada entre ambos algoritmos. Burbuja requiere cada vez más tiempo conforme aumenta la cantidad de elementos. Con 10.000 elementos obtuvo un promedio de 679,56 ms, mientras que con 80.000 elementos alcanzó 34.856,32 ms, aproximadamente 34,86 segundos.

QuickSort, en cambio, mantuvo tiempos mucho menores. Para 10.000 elementos obtuvo 2,04 ms y para 80.000 elementos 14,90 ms. Aunque los tiempos pueden variar ligeramente entre ejecuciones debido a las condiciones del equipo, la diferencia de rendimiento entre ambos algoritmos es clara.

El comportamiento observado coincide con el análisis teórico: el crecimiento cuadrático de Burbuja hace que el costo aumente rápidamente con n, mientras que el comportamiento promedio O(n log n) de QuickSort permite manejar cantidades mayores de datos de forma mucho más eficiente.

VERIFICACIÓN DEL ORDENAMIENTO

El programa incluye una función de verificación que recorre el arreglo ordenado y comprueba que cada elemento sea menor o igual al siguiente. De esta manera, no se mide únicamente el tiempo, sino que también se comprueba que el resultado sea correcto.

La verificación se realiza después de cada corrida de Burbuja y QuickSort.

CONCLUSIONES

- La práctica permitió comprobar experimentalmente la diferencia de rendimiento entre un algoritmo $O(n^2)$ y uno O(n log n) en promedio.

- Burbuja presentó un incremento muy pronunciado del tiempo de ejecución al aumentar la cantidad de elementos, confirmando el impacto de su complejidad cuadrática.
- QuickSort presentó un rendimiento considerablemente superior en todas las cantidades evaluadas.
- La elección del algoritmo tiene una influencia importante en el rendimiento de un programa, especialmente cuando se trabaja con grandes cantidades de datos.
- El uso de calentamiento, tres corridas y promedio permite obtener una medición más representativa que una única ejecución.
- La verificación posterior confirmó que ambos algoritmos dejaron los arreglos correctamente ordenados.

ANEXO:

código principal utilizado

```
using System;
using System.Diagnostics;

class Program
{
    static void Main()
    {
        Console.WriteLine("ANALISIS DE ALGORITMOS - C5");

        int[] cantidades = { 10000, 20000, 40000, 80000 };

        double[] resultadosBurbuja = new double[cantidades.Length];
        double[] resultadosQuickSort = new double[cantidades.Length];

        for (int i = 0; i < cantidades.Length; i++)
        {
            int cantidad = cantidades[i];
            int[] datos = GenerarDatos(cantidad);

            // Calentamiento
            int[] calentamientoBurbuja = (int[])datos.Clone();
            BubbleSort(calentamientoBurbuja);

            int[] calentamientoQuickSort = (int[])datos.Clone();
            QuickSort(calentamientoQuickSort, 0,
                      calentamientoQuickSort.Length - 1);

            // Tres corridas y promedio
            resultadosBurbuja[i] = MedirBurbuja(datos);
            resultadosQuickSort[i] = MedirQuickSort(datos);
        }

        MostrarTabla(cantidades,
                     resultadosBurbuja,
                     resultadosQuickSort);
    }
```

```
static int[] GenerarDatos(int cantidad)
{
    Random aleatorio = new Random(12345);
    int[] datos = new int[cantidad];

    for (int i = 0; i < cantidad; i++)
        datos[i] = aleatorio.Next(1, 1000000);

    return datos;
}

static double MedirBurbuja(int[] datos)
{
    double suma = 0;

    for (int corrida = 1; corrida <= 3; corrida++)
    {
        int[] copia = (int[])datos.Clone();

        Stopwatch reloj = Stopwatch.StartNew();
        BubbleSort(copia);
        reloj.Stop();

        double tiempo = reloj.Elapsed.TotalMilliseconds;
        Console.WriteLine($"Burbuja - Corrida {corrida}: {tiempo:F2} ms");

        if (!EstaOrdenado(copia))
            Console.WriteLine("ERROR: Burbuja no ordenó correctamente.");

        suma += tiempo;
    }

    return suma / 3;
}

static double MedirQuickSort(int[] datos)
{
    double suma = 0;

    for (int corrida = 1; corrida <= 3; corrida++)
    {
        int[] copia = (int[])datos.Clone();

        Stopwatch reloj = Stopwatch.StartNew();
        QuickSort(copia, 0, copia.Length - 1);
        reloj.Stop();

        double tiempo = reloj.Elapsed.TotalMilliseconds;
        Console.WriteLine($"QuickSort - Corrida {corrida}: {tiempo:F2} ms");

        if (!EstaOrdenado(copia))
```

```
            Console.WriteLine("ERROR: QuickSort no ordenó correctamente.");

        suma += tiempo;
    }

    return suma / 3;
}

static void BubbleSort(int[] arreglo)
{
    int n = arreglo.Length;

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arreglo[j] > arreglo[j + 1])
            {
                int temporal = arreglo[j];
                arreglo[j] = arreglo[j + 1];
                arreglo[j + 1] = temporal;
            }
        }
    }
}

static void QuickSort(int[] arreglo, int izquierda, int derecha)
{
    if (izquierda < derecha)
    {
        int posicionPivote =
            Particionar(arreglo, izquierda, derecha);

        QuickSort(arreglo, izquierda, posicionPivote - 1);
        QuickSort(arreglo, posicionPivote + 1, derecha);
    }
}

static int Particionar(int[] arreglo, int izquierda, int derecha)
{
    int pivote = arreglo[derecha];
    int i = izquierda - 1;

    for (int j = izquierda; j < derecha; j++)
    {
        if (arreglo[j] <= pivote)
        {
            i++;

            int temporal = arreglo[i];
            arreglo[i] = arreglo[j];
            arreglo[j] = temporal;
        }
    }
```

```
        int temporal2 = arreglo[i + 1];
        arreglo[i + 1] = arreglo[derecha];
        arreglo[derecha] = temporal2;

        return i + 1;
    }

    static bool EstaOrdenado(int[] arreglo)
    {
        for (int i = 0; i < arreglo.Length - 1; i++)
        {
            if (arreglo[i] > arreglo[i + 1])
                return false;
        }

        return true;
    }

    static void MostrarTabla(
        int[] cantidades,
        double[] burbuja,
        double[] quickSort)
    {
        Console.WriteLine("\nTABLA DE TIEMPOS");

        for (int i = 0; i < cantidades.Length; i++)
        {
            Console.WriteLine(
                $"{cantidades[i]} | " +
                $"Burbuja: {burbuja[i]:F2} ms | " +
                $"QuickSort: {quickSort[i]:F2} ms");
        }
    }
}
```